# Directed Graph Search Algorithms

Kanishk Goel

14 November 2025

## 1 Dijkstra

**Single-Source Shortest Paths**

**Input:** A directed graph $G = (V, E)$, a starting vertex $s \in V$, and a nonnegative length $l_e$ for each edge $e \in E$
**Output:** $dist(s, v)$ for every vertex $v \in V$

**Dijkstra**

**Input:** directed graph $G = (V, E)$ in adjacency-list representation, a vertex $s \in V$, a length $l_e \geq 0$ for each $e \in E$
**Output:** for every $v$, the value $len(v)$ equals the true shortest-path distance $dist(s, v)$.

---

`// Initialization`
$X \leftarrow \{s\}$
$len(s) \leftarrow 0$, $len(s) \leftarrow$ inf, for every $v \neq s$
**while** there is an edge $(v, w)$ with $v \in X, w \notin X$ **do**
  $(v^*, w^*) \leftarrow$ such an edge minimizing $len(v) + l_{vw}$
  add $w^*$ to $X$
  $len(w^*) \leftarrow len(v^*) + l_{v^*w^*}$

Runtime being $O(mn)$ if heap is not used.

**Dijkstra**

**Input:** directed graph $G = (V, E)$ in adjacency-list representation, a vertex $s \in V$, a length $l_e \geq 0$ for each $e \in E$

**Output:** for every $v$, the valye $len(v)$ equals the true shortest-path distance $dist(s, v)$.

---

```
// Initialization
X ← {}, H ← empty heap
key(s) ← 0
foreach v ≠ s do
    key(v) ← ∞
foreach v ∈ V do
    Insert v into H   // or use Heapify
// Main Loop
while H is non-empty do
    w* ← ExtractMin(H)
    add w* to X
    len(w*) ← key(w*)
    // update heap to maintain invariant
    foreach edge (w*, y) do
        Delete y from H
        key(y) ← min{key(y), len(w*) + l_{w*y}}
        Insert y into H
```

Runtime being $O((m + n) \log n)$ if heap is used.